

ENGINEERING the Conditions for Knowledge

A conceptual and engineering foundation for
SymK

**Knowledge Engineering does not manufacture
Knowledge.
It engineers the conditions for responsible epistemic
achievement.**

Foundational Working Paper - Version 0.1
14 August 2026

Paper status and central proposition

This paper defines the discipline implied when SymK calls itself an Engineering of Knowledge System. It is a durable working reference for conceptual and technical development, not a final ratified specification.

Knowledge Engineering builds and governs accountable, evolvable conditions for epistemic achievement. It does not manufacture truth or turn representations into Knowledge by declaration.

The paper inherits the expert-systems and model-based traditions, then expands the engineering boundary to the complete epistemic lifecycle: inquiry, representation, challenge, application, consequence, Education, and revision.

The working definition and twelve commitments are candidates for constitutional review. Publication does not silently ratify them as canonical SymK axioms.

Abstract

SymK describes itself as an Engineering of Knowledge System. That description cannot remain an attractive phrase. It must distinguish what is engineered, what is not, and what responsibilities arise when technical and institutional systems mediate Knowledge among Intelligences.

The classical account defines Knowledge Engineering as the discipline of building systems that acquire, represent, reason with, and apply expert knowledge. Emerging from artificial-intelligence research and expert systems, it confronted the difficulty of eliciting expertise, formalizing it, validating a knowledge base, and maintaining useful performance. Later methodologies replaced the naive idea of transferring knowledge from an expert's head with the construction of explicit models of tasks, domains, inferences, organizations, and communication.

These achievements remain essential but insufficient for SymK. The Knowledge foundation established that Knowledge is an epistemic achievement, not a document, rule, graph, model weight, or database entry. An engineered system cannot manufacture truth by assertion or convert representation into Knowledge by naming it so. Nor does Knowledge Engineering merely bridge what humans know and computers process: relevant Knowledge may be practical, collective, institutional, artificial, contested, embodied, or realized in a hybrid system.

This paper therefore proposes a broader working definition: Knowledge Engineering is the disciplined design, construction, governance, evaluation, and evolution of the conditions through which Intelligences and cooperative systems can pursue, achieve, express, preserve, share, challenge, apply, and revise Knowledge responsibly within stated scopes. Its direct artifacts are not Knowledge itself but epistemic processes, roles, representations, provenance structures, validation arrangements, interfaces, controls, and environments that support or impair Knowledge.

The paper distinguishes Knowledge Engineering from epistemology, data engineering, information management, software engineering, AI engineering, knowledge management, and education. It develops a layered model, rejects the reduction of data to facts and Knowledge to computable meaning, treats tacit Knowledge as a limit on extraction, examines generative AI as a relocation rather than elimination of the acquisition bottleneck, proposes twelve candidate foundational commitments, and derives engineering requirements and an evaluation gate.

Its central conclusion is:

Knowledge Engineering does not engineer truth or Knowledge directly. It engineers accountable and evolvable conditions under which epistemic achievements can arise, become shareable, guide action, survive challenge, and remain open to correction.

Contents

Abstract	3
1. Why Knowledge Engineering is constitutional to SymK	5
2. The inherited definition and what it gets right	5
3. Historical development: from expert systems to model-based engineering	6
4. The object problem: what can actually be engineered?	7
5. The central correction: Knowledge Engineering engineers epistemic conditions	8
6. Acquisition is construction, not extraction	9
7. Representation is projection, not preservation without remainder	9
8. Reasoning, validation, and application	10
9. Distinctions SymK must preserve	11
10. A working definition for SymK	12
11. The SymK epistemic lifecycle	13
12. Candidate foundational commitments	13
13. Implications for Human-AI and multi-Intelligence cooperation	14
14. Development requirements derived from the foundation	15
15. Open foundational questions	16
16. A practical Knowledge Engineering gate	17
17. Conclusion	18
References	19
Document governance note	19

1. Why Knowledge Engineering is constitutional to SymK

SymK began as a protocol for symbiotic cooperation between Human Intelligence and Artificial Intelligence. It expanded into an Engineering of Knowledge System and generalized its objective toward cooperation among Intelligences of any kind.

Three foundations now constrain that evolution:

- Intelligence is a substrate-realized capacity of an organized system, not a material component or isolated performance.
- Knowledge is an epistemic achievement in which an Intelligence or cooperative system stands in a truth-connected, competence-bearing, or direct relation to what is known.
- Education forms Intelligences for responsible participation, while Training gives relevant dispositions durable expression in practice.

Knowledge Engineering connects these foundations to deliberate construction. It asks how systems can support cumulative cooperation when participants differ in substrate, capability, representation, authority, time, vocabulary, and access to evidence.

If the concept is left undefined, SymK risks inheriting several reductions:

1. Knowledge Engineering becomes ontology construction.
2. Stored claims become Knowledge because a database calls them facts.
3. Human expertise is treated as a substance that can be extracted and transferred.
4. Formal validity is mistaken for truth or responsible applicability.
5. Model output is mistaken for epistemic achievement.
6. Technical validation is substituted for domain validation.
7. Knowledge management is reduced to moving documents.
8. Governance is added after the representational architecture has already fixed power and meaning.
9. Tacit, practical, embodied, and collective Knowledge disappear because they resist a chosen formalism.
10. The system's effects on participants and institutions remain outside the engineering boundary.

SymK needs a concept wide enough to govern the complete epistemic lifecycle and precise enough to guide implementation.

2. The inherited definition and what it gets right

A strong contemporary summary states:

Knowledge Engineering is the discipline concerned with building systems that represent, reason about, and apply knowledge, rather than merely data or information.

It usually identifies four practices:

- knowledge acquisition from experts, documents, observations, or experience;
- knowledge representation through rules, frames, logic, ontologies, graphs, cases, or models;

- reasoning through deduction, classification, diagnosis, planning, analogy, or uncertain inference;
- validation against expert judgment, formal constraints, cases, and operational outcomes.

This account correctly recognizes that syntax alone is insufficient. A field value acquires operational meaning through a model, context, task, and interpretive community. It correctly identifies tacit expertise as difficult to articulate, representation as selective, inference as dependent on assumptions, and validation as more than software execution.

It also preserves an important practical identity. Knowledge Engineering is not merely philosophical reflection. It builds artifacts that must work.

But three claims require correction for SymK:

1. The central gap is not only between human Knowledge and machine processing.
2. Making Knowledge explicit and computable does not preserve it automatically.
3. Data engineering does not simply manage facts while Knowledge Engineering manages meaning.

These corrections do not reject the inherited discipline. They clarify its object and enlarge its boundary.

3. Historical development: from expert systems to model-based engineering

3.1 The expert-systems origin

In the 1960s and 1970s, AI researchers increasingly found that strong performance in specialized tasks depended less on a universal reasoning procedure than on extensive domain-specific structure. Systems such as DENDRAL and MYCIN combined encoded expertise with inference mechanisms. Feigenbaum described Knowledge Engineering as bringing AI principles and tools to difficult application problems requiring expert Knowledge, emphasizing acquisition, representation, use, and explanation [Feigenbaum 1977].

The knowledge engineer occupied a mediating role between specialist and machine. The engineer elicited rules, categories, exceptions, and strategies; encoded them; observed failures; and refined the system.

3.2 The acquisition bottleneck

The difficulty of constructing knowledge bases became known as the knowledge-acquisition bottleneck. Experts often omit what has become obvious to them, disagree with one another, rationalize decisions retrospectively, or rely on perceptual and practical discriminations they cannot state as rules. Interviews produce accounts of practice, not transparent copies of cognitive mechanisms.

The bottleneck was therefore not only one of labor. It exposed a faulty metaphor: expertise is not a container whose contents can simply be transferred.

3.3 From transfer to modeling

By the 1980s and 1990s, Knowledge Engineering increasingly adopted a modeling view. Studer, Benjamins, and Fensel describe this change explicitly: a knowledge base is not a reproduction of expert cognition but an engineered model constructed for a task [Studer, Benjamins, and Fensel 1998].

Methodologies such as KADS and CommonKADS separated organizational, task, agent, knowledge, communication, and design models. Problem-solving methods distinguished reusable inference structures from domain-specific content. MIKE combined informal elicitation, semiformal models, formal specification, prototyping, and iterative development.

This was a decisive maturation. It made assumptions, abstractions, roles, and task structures more visible and integrated Knowledge Engineering with software and organizational analysis.

3.4 Ontologies and shared conceptualization

Ontology engineering shifted attention from isolated expert systems toward shared vocabularies and reusable conceptual models. Gruber defined an ontology, in the computational sense, as an explicit specification of a conceptualization [Gruber 1993]. Description logics and OWL brought formal semantics and computationally controlled reasoning to concept and relation models [W3C 2012].

The achievement is substantial, but a shared conceptualization remains a model. Agreement on vocabulary does not establish that every assertion is true or every category just.

3.5 Knowledge graphs and integration

Knowledge graphs organize entity-relation assertions and support identity, linking, traversal, enrichment, and inference. They have become central to search, recommendation, scientific data, enterprise integration, and AI grounding [Hogan et al. 2021].

Yet the term knowledge graph can conceal the central distinction. A graph may contain supported claims, disputed claims, obsolete classifications, generated hypotheses, or errors. Graph structure does not confer Knowledge.

3.6 Machine learning and generative systems

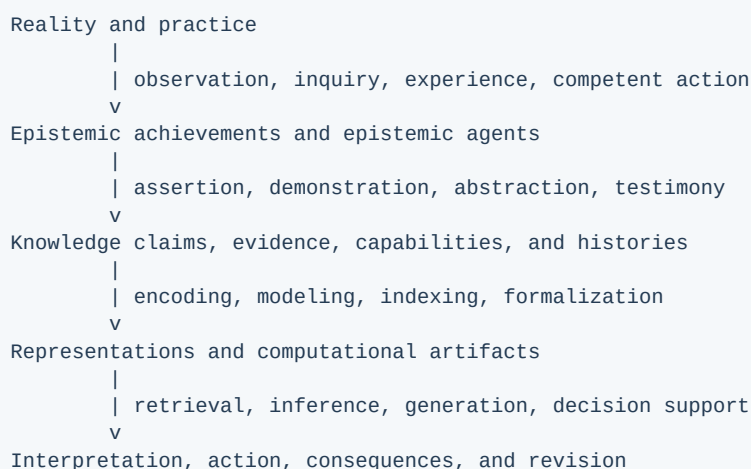
Machine learning shifted representation from predominantly explicit symbols toward parameters learned from examples. Generative models now synthesize, transform, retrieve, classify, and propose structures at unprecedented scale.

This does not make Knowledge Engineering obsolete. It changes its materials and risks. The engineer must now govern datasets, learned representations, prompts, retrieval contexts, tools, evaluations, generated claims, human review, and downstream feedback. The acquisition bottleneck is partly relocated from manual rule encoding to source selection, model formation, evaluation, grounding, adjudication, and maintenance.

4. The object problem: what can actually be engineered?

Engineering normally transforms materials and organizations under constraints toward purposes. If Knowledge is an epistemic achievement rather than an artifact, what is the material of Knowledge Engineering?

SymK distinguishes five levels:



Knowledge Engineering acts upon every transition but owns none automatically.

It can engineer:

- inquiry and acquisition procedures;
- roles, responsibilities, and challenge rights;
- claim, evidence, and provenance structures;
- vocabularies, models, rules, graphs, and interfaces;
- validation and evaluation processes;
- memory, versioning, and supersession mechanisms;
- access, privacy, authorization, and accountability controls;
- environments for Education, Training, deliberation, and repair;
- and feedback from action and consequence into revision.

It cannot directly engineer:

- truth by institutional decree;
- Knowledge merely by storing a representation;
- understanding merely by producing an explanation-shaped output;
- competence merely by documenting a procedure;
- consensus merely by deleting disagreement;
- or responsible reliance merely by assigning a confidence score.

This is the ontological humility required by the discipline.

5. The central correction: Knowledge Engineering engineers epistemic conditions

The phrase engineering Knowledge admits two readings.

The strong reading says Knowledge itself is manufactured as an engineered product. This conflicts with the SymK Knowledge foundation. Truth is not created by governance, and practical competence cannot be produced by representation alone.

The disciplined reading says engineering constructs conditions that make epistemic achievements more possible, communicable, durable, testable, usable, and revisable.

SymK adopts the disciplined reading:

Knowledge Engineering does not engineer truth or Knowledge directly. It engineers accountable and evolvable conditions under which epistemic achievements can arise, become shareable, guide action, survive challenge, and remain open to correction.

This is not a retreat from engineering. Medicine does not manufacture health directly; it designs interventions and institutions that act on conditions of health. Education does not insert understanding; it organizes formative conditions. Likewise, Knowledge Engineering works causally and materially without claiming sovereignty over its epistemic outcome.

The quality of Knowledge Engineering is therefore evaluated not only by representational elegance or system accuracy, but by the epistemic ecology it produces.

6. Acquisition is construction, not extraction

6.1 Experts are participants, not containers

An expert's statement is evidence about expertise, not an unmediated export of Knowledge. Elicitation is an interaction shaped by questions, examples, vocabulary, status, memory, incentives, and context.

The engineer also contributes assumptions by selecting what to ask, what to omit, how to abstract, and which disagreements to resolve. Knowledge acquisition is therefore better understood as model construction under epistemic constraints.

6.2 Tacit Knowledge is not merely undocumented text

Polanyi's observation that people can know more than they can tell identifies a genuine limit [Polanyi 1966]. Practical Knowledge may be embodied in timing, perception, skilled movement, situated attention, or participation in a practice.

Making some aspect explicit can improve teaching, coordination, and computation. But externalization is a transformation, not lossless decompression. The resulting procedure, ontology, example set, or model may support the competence without becoming identical to it.

6.3 Documents are testimony and artifacts

Documents preserve claims, instructions, observations, decisions, and evidence. They can participate in Knowledge but do not certify themselves. Acquisition from documents must preserve authorship, purpose, time, jurisdiction, version, audience, and relation to practice.

6.4 Data-driven acquisition

Statistical learning discovers regularities in observations selected and represented through pipelines. These regularities may support prediction or reveal structure, but the dataset is neither raw reality nor a neutral collection of facts. Measurement, sampling, labeling, missingness, and historical action shape it.

6.5 Generative acquisition

Generative AI can propose concepts, rules, mappings, summaries, and questions rapidly. It can expose omissions and compare representations. It also generates plausible but unsupported structures, normalizes majority patterns, and obscures source boundaries.

Generative systems can reduce the cost of producing candidates. They do not eliminate the cost of establishing meaning, fit, authority, evidence, and consequence.

Generative AI accelerates proposition. It does not automate epistemic warrant.

7. Representation is projection, not preservation without remainder

Every representation selects. A rule foregrounds conditional structure; a graph foregrounds entities and relations; a statistical model foregrounds learned regularities; a document foregrounds narrative or argument; a workflow foregrounds action sequence.

The Properties and Projections foundation supplies the appropriate principle:

Objects are described through properties, and classifications are purpose-dependent projections over selected properties.

Knowledge representations are likewise projections. Their value depends on purpose, users, reasoning tasks, scale, evidence, and consequence.

7.1 Formal semantics are local guarantees

A logic can establish that a conclusion follows from premises under specified semantics. It does not establish that the premises accurately describe reality, that the vocabulary is appropriate, or that the conclusion should govern action.

7.2 Computability introduces tradeoffs

Expressiveness, decidability, performance, explainability, maintainability, and interoperability constrain one another. A representation faithful to one task may be unusable for another.

7.3 Multiple representations may be necessary

Propositions, procedures, examples, cases, demonstrations, simulations, narratives, measurements, and models may express complementary aspects of a domain. SymK should support governed relations among them rather than demand one universal form.

7.4 Meaning is enacted as well as encoded

Meaning depends not only on a schema definition but on use, interpretation, institutions, and consequences. Two systems can share identifiers while applying them differently. Interoperability is therefore an achieved relation, not a property of syntax alone.

8. Reasoning, validation, and application

8.1 Reasoning

Reasoning mechanisms transform representations under rules, algorithms, learned patterns, or mixed procedures. SymK must preserve:

- premises and source versions;
- inference method and assumptions;
- uncertainty and alternatives;
- tools, models, and participants involved;
- scope in which the result is licensed;
- and whether the output is a deduction, prediction, hypothesis, recommendation, or decision.

An inferred claim is not epistemically self-sufficient merely because the computation is correct.

8.2 Validation is plural

Different objects require different validation:

- syntax and schema validation;
- logical consistency or satisfiability checks;
- test cases and benchmark performance;

- expert review;
- empirical comparison with observations;
- usability and task-fit evaluation;
- inter-rater or inter-system disagreement analysis;
- operational monitoring;
- legal and ethical review;
- and consequence assessment.

No single validator settles all dimensions. Expert consensus can be wrong; formal consistency can preserve false premises; empirical success can exploit a shortcut; user satisfaction can reward confident error.

8.3 Application closes the epistemic loop

Application changes the world from which future evidence is drawn. A diagnostic classification changes treatment; a legal classification changes rights; a recommendation changes exposure; an automated decision changes behavior.

Knowledge Engineering must therefore preserve the transition from representation to action and feed consequences back into review. Deployment is not the end of the knowledge lifecycle.

9. Distinctions SymK must preserve

9.1 Knowledge Engineering and epistemology

Epistemology investigates Knowledge, warrant, truth, belief, competence, testimony, disagreement, and related concepts. Knowledge Engineering builds and governs systems that mediate epistemic activity. It needs epistemology but cannot replace philosophical questions with schema fields.

9.2 Knowledge Engineering and data engineering

Data engineering builds reliable systems for collecting, transforming, storing, and delivering data. It does not merely manage facts. Data may be measured, inferred, labeled, erroneous, synthetic, or disputed. Meaning and governance already enter data models, pipelines, and quality rules.

Knowledge Engineering differs in emphasis: it makes claims, concepts, grounds, inference, competence, context, and responsible application explicit. The boundary is overlapping, not metaphysical.

9.3 Knowledge Engineering and information management

Information management organizes resources for access, retention, security, and use. Knowledge Engineering additionally models epistemic relations, reasoning, grounds, capability, contradiction, and revision. A well-managed document can still express a poorly supported claim.

9.4 Knowledge Engineering and software engineering

Software engineering constructs dependable computational systems. Knowledge Engineering must use it, but adds a specific concern with domain models, epistemic content, reasoning assumptions, acquisition, and validation. A system can be technically correct and epistemically defective.

9.5 Knowledge Engineering and AI engineering

AI engineering builds systems using learning, inference, planning, perception, or generative models. Some AI systems are knowledge-intensive; others optimize patterns without explicit epistemic structures. Knowledge Engineering asks how contributions become warranted, contextualized, governed, and revisable regardless of whether AI is used.

9.6 Knowledge Engineering and knowledge management

Knowledge management concerns organizational creation, sharing, retention, and use of knowledge resources and capabilities. Knowledge Engineering supplies modeling and system-development methods but should not absorb all organizational practice into technology.

9.7 Knowledge Engineering and Education

Knowledge Engineering designs conditions and artifacts for epistemic work. Education forms Intelligences for responsible participation in that work. Each depends on the other: systems train habits, while educated participants are necessary to challenge and improve systems.

10. A working definition for SymK

Knowledge Engineering, in SymK, is the disciplined design, construction, governance, evaluation, and evolution of the conditions through which Intelligences and cooperative systems can pursue, achieve, express, preserve, share, challenge, apply, and revise Knowledge responsibly within stated scopes.

Disciplined

The work is explicit about purposes, methods, evidence, assumptions, roles, limitations, and consequences. Craft judgment remains necessary but becomes inspectable.

Design and construction

Knowledge Engineering creates real structures: processes, vocabularies, models, software, interfaces, roles, institutions, and controls.

Governance

Because representation and access allocate power, authority, challenge, privacy, and responsibility are architectural concerns from the beginning.

Evaluation and evolution

Knowledge systems age. Sources change, models drift, concepts become contested, laws supersede one another, and application reveals unforeseen effects. Revision is normal operation.

Conditions

This word marks the ontological boundary. Knowledge Engineering acts on environments and relations that support epistemic achievement; it does not declare the achievement into existence.

Intelligences and cooperative systems

The relevant participants may be human, artificial, social, collective, or hybrid. Epistemic roles must be attributed from evidence rather than assigned permanently by origin.

The epistemic verbs

- pursue through inquiry and question formation;
- achieve through truth-connected, competence-bearing, or direct relations;

- express through claims, demonstrations, procedures, and representations;
- preserve through memory, provenance, and lineage;
- share through translation and accessible forms;
- challenge through counterevidence, disagreement, and review;
- apply through interpretation and action;
- revise through correction, supersession, and learning.

Responsibly within stated scopes

Engineering quality includes fitness for reliance, affected parties, authorization, uncertainty, and limits. No model or claim is universally applicable merely because it is technically available.

11. The SymK epistemic lifecycle

Knowledge Engineering should govern a lifecycle rather than a static repository:

1. Purpose and question formation - What is being attempted, for whom, and why?
2. Boundary and participant identification - Which systems, communities, practices, and consequences matter?
3. Inquiry and acquisition - Which observations, expertise, documents, datasets, and methods contribute?
4. Elicitation and interpretation - How are contributions translated, compared, and contextualized?
5. Modeling and representation - Which projections serve the required tasks?
6. Validation and challenge - What supports, contradicts, or limits the claims and capabilities?
7. Operational acceptance - Who may rely on what, within which scope and authority?
8. Application and action - How does the representation enter decisions or practice?
9. Consequence observation - What occurred, including unintended effects?
10. Revision, supersession, and retirement - What must change, remain contested, or stop being used?
11. Education and institutional memory - What must participants and the cooperative system learn?

The lifecycle is recursive. Consequences generate new questions; challenges reshape models; Education changes the capacities available for later inquiry.

12. Candidate foundational commitments

KE-01 - Knowledge and engineered artifacts are distinct

Documents, rules, graphs, ontologies, models, and databases may support Knowledge but are not Knowledge merely by existing.

KE-02 - Knowledge Engineering engineers conditions

The discipline constructs processes, relationships, representations, controls, and environments that enable epistemic achievement and responsible reliance.

KE-03 - Acquisition is constructive and situated

Elicitation, observation, and learning transform contributions through questions, purposes, abstractions, and representational choices.

KE-04 - Representation is a governed projection

Every representation selects dimensions for a purpose and must expose what it suppresses, assumes, or cannot express.

KE-05 - Formal validity is not epistemic sufficiency

Correct inference depends on premises, semantics, scope, and relevance that require independent evaluation.

KE-06 - Tacit Knowledge cannot be presumed extractable

Procedures and models may scaffold competence without exhausting the practical, embodied, or situated Knowledge from which they were developed.

KE-07 - Validation is plural and continuing

Formal, empirical, expert, operational, social, legal, and consequential evaluations answer different questions. Deployment does not end validation.

KE-08 - Contradiction is represented before resolution

Disagreement may reveal scope, perspective, uncertainty, change, or power. Storage convenience must not manufacture false consensus.

KE-09 - Provenance and context are part of the durable system

SymK preserves how, why, when, by whom, under what authority, and within which conditions a contribution arose.

KE-10 - Epistemic role is not fixed by Intelligence type

Humans, machines, organizations, and hybrid systems may propose, test, formalize, criticize, remember, or apply. Their standing depends on the role, process, evidence, and scope.

KE-11 - Application and consequence belong to the knowledge lifecycle

The effects of relying on a model or claim provide evidence for its revision and for the redesign of the system.

KE-12 - Knowledge Engineering is inseparable from governance and Education

Architectures allocate authority and train habits. Responsible systems govern formative power, challenge rights, and the continuing development of participants.

13. Implications for Human-AI and multi-Intelligence cooperation

13.1 AI output is a candidate contribution

A generated statement, mapping, rule, or ontology class enters the lifecycle as a contribution with provenance and uncertainty. Fluency, confidence, or model scale does not confer epistemic acceptance.

13.2 Human approval is not a universal warranty

Human reviewers have limited time, attention, domain coverage, and independence. Review quality must itself be modeled through role, competence, evidence, procedure, and disagreement.

13.3 The bearer may be hybrid

A useful epistemic capability may arise from a human, model, retrieval system, corpus, validation service, interface, and institution acting together. Evaluation should target the organized system, not attribute the whole achievement to its most visible component.

13.4 Translation is epistemic work

Moving between legal language, ontology terms, database fields, model embeddings, and user explanations is not formatting alone. Translation changes distinctions and may introduce or remove ambiguity.

13.5 Generative AI relocates the bottleneck

Candidate production becomes abundant. Scarcity moves toward framing, source quality, validation, adjudication, integration, responsibility, and maintenance. Knowledge Engineering becomes more necessary as generation becomes cheaper.

13.6 Cooperation can create new Knowledge

Participants may combine evidence, capabilities, and perspectives so that no participant independently contains the full achievement. SymK must preserve distributed grounds and collective responsibility without pretending that consensus is truth.

14. Development requirements derived from the foundation

14.1 Model epistemic objects separately

At minimum, distinguish:

- observations and source artifacts;
- claims and questions;
- evidence and grounds;
- counterevidence and challenges;
- agents, roles, and competencies;
- methods and inference episodes;
- representations and projections;
- decisions and authorizations;
- actions and consequences;
- epistemic status, uncertainty, and scope;
- versions, supersession, and retirement.

14.2 Preserve lineage across transformations

Extraction, summarization, translation, classification, inference, and generation should create traceable derived objects rather than silently replacing sources.

14.3 Make assumptions executable or inspectable

Where possible, assumptions should be testable constraints. Where they cannot be formalized, they should remain visible as governed statements.

14.4 Support multiple models and projections

No ontology, graph, folder structure, embedding space, or model should become the domain by implementation accident. SymK must relate alternate representations to purposes and evidence.

14.5 Treat uncertainty as typed

Distinguish missing information, measurement error, model uncertainty, disagreement, ambiguity, stochastic variation, and future contingency. One confidence number is rarely sufficient.

14.6 Build challenge into normal operation

Participants need mechanisms to dispute claims, representations, classifications, inference rules, source relevance, and application. Challenges require status, evidence, response, and history.

14.7 Validate at multiple layers

Test formal correctness, semantic fit, empirical adequacy, task usefulness, usability, security, fairness, legality, and consequences independently where relevant.

14.8 Observe use and non-use

The system should record where an asset was applied, rejected, bypassed, misunderstood, or harmful. Absence of use can reveal poor fit or institutional resistance.

14.9 Govern generative contributions

Generated candidates require source boundaries, model and prompt provenance, validation state, reviewer identity, and prohibition against silent promotion into accepted Knowledge assets.

14.10 Support practical Knowledge

Represent procedures together with demonstrations, capability evidence, supervision conditions, failure modes, and transfer limits. A procedure document is not competence.

14.11 Integrate Education

Interfaces and workflows should cultivate provenance, uncertainty disclosure, fair challenge, correction, and repair. System design inevitably trains participant habits.

14.12 Design for retirement

Every maintained model, rule, classification, and accepted claim needs review conditions and a path to deprecation without destroying lineage.

15. Open foundational questions

15.1 Is Knowledge Engineering the right universal name?

The phrase may suggest direct manufacture of Knowledge. SymK can retain it if the conditions interpretation remains explicit; alternatives include epistemic systems engineering or engineering for Knowledge.

15.2 What is the minimum epistemic object?

Claim, source, ground, scope, agent, and status are strong candidates, but practical Knowledge and acquaintance may require different primitives.

15.3 Can an artificial system be a knowledge engineer?

It may perform acquisition, modeling, inference, and validation tasks. Whether it bears responsibility or merely participates as a governed component depends on agency, authority, accountability, and system organization.

15.4 What remains outside representation?

Tacit, embodied, affective, and situated dimensions may resist durable encoding. SymK needs methods to acknowledge and work with them without fictional completeness.

15.5 How should incompatible ontologies coexist?

Alignment may be partial, asymmetric, task-relative, or politically contested. Forced unification can destroy meaningful difference.

15.6 When does validation justify reliance?

The required evidence depends on consequence, reversibility, domain, novelty, and affected parties. SymK needs proportional assurance models.

15.7 How should collective epistemic responsibility be allocated?

Distributed contributions complicate authorship, error correction, authority, and liability. The whole may know or act without any member holding the complete grounds.

15.8 Can the knowledge lifecycle govern itself?

Rules for acceptance and revision are themselves knowledge claims and institutional choices. SymK requires recursive governance without infinite regress or unchallengeable authority.

16. A practical Knowledge Engineering gate

Before a SymK initiative is described as Knowledge Engineering, ask:

Dimension	Question
Purpose	Which epistemic or cooperative problem is being addressed, for whom, and why?
Bearer	Which individual, collective, artificial, or hybrid system may know or act?
Sources	Which observations, experts, documents, datasets, practices, and models contribute?
Acquisition	How were contributions elicited, selected, transformed, or learned?
Claims	What is asserted, hypothesized, prescribed, or demonstrated?
Grounds	What evidence, competence, method, or direct relation supports reliance?
Representation	Which projection is used, for which task, and what does it suppress?
Inference	Which premises, rules, models, assumptions, and tools produce conclusions?
Validation	Which formal, empirical, expert, operational, and consequential checks apply?
Scope	Where, when, for whom, and under which conditions is reliance licensed?
Uncertainty	What is unknown, disputed, ambiguous, variable, or model-dependent?
Governance	Who may propose, accept, challenge, authorize, revise, and retire?
Application	How does the asset enter interpretation, decision, or action?

Dimension	Question
Consequences	What effects occurred, including harms and unintended learning?
Evolution	How are versioning, supersession, contradiction, decay, and retirement handled?
Education	Which habits and capabilities does the system form in its participants?

Passing this gate does not certify that Knowledge has been achieved. It establishes that the system responsibly engineers the conditions, evidence, representations, processes, and governance relevant to the claim.

17. Conclusion

Knowledge Engineering began as the art of building AI systems around expert Knowledge. It discovered that expertise was difficult to elicit, that representation shaped what could be reasoned about, and that useful systems required continuing refinement. Mature methodologies replaced extraction with modeling and expanded the engineering boundary to tasks, organizations, agents, communication, and lifecycle.

SymK inherits that discipline and subjects it to its own Knowledge foundation. If Knowledge is an epistemic achievement, no knowledge base, ontology, graph, model, or generated answer is Knowledge merely because it is structured or useful. Engineering must preserve the distinction among reality, epistemic achievement, claim, representation, inference, application, and consequence.

The proposed definition is:

Knowledge Engineering, in SymK, is the disciplined design, construction, governance, evaluation, and evolution of the conditions through which Intelligences and cooperative systems can pursue, achieve, express, preserve, share, challenge, apply, and revise Knowledge responsibly within stated scopes.

Its limiting principle is:

Knowledge Engineering does not engineer truth or Knowledge directly. It engineers accountable and evolvable conditions under which epistemic achievements can arise, become shareable, guide action, survive challenge, and remain open to correction.

This changes the center of the discipline. Its primary problem is no longer only how to translate human expertise into forms computers can process. It is how heterogeneous Intelligences can build and inhabit epistemic systems worthy of responsible reliance.

Within SymK:

Knowledge is the shared medium. Cooperation is the objective. Education forms the participants. Knowledge Engineering builds and governs the conditions through which their epistemic cooperation can become cumulative, actionable, and self-correcting.

References

- Angele, J., Fensel, D., Landes, D., and Studer, R. (1998). "Developing Knowledge-Based Systems with MIKE." *Automated Software Engineering*, 5, 389-418. [Publisher record](#).
- Buchanan, B. G., and Shortliffe, E. H., eds. (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley. [Open text](#).
- Feigenbaum, E. A. (1977). "The Art of Artificial Intelligence: Themes and Case Studies of Knowledge Engineering." *Proceedings of IJCAI-77*. [Archival record](#).
- Forsythe, D. E. (1993). "Engineering Knowledge: The Construction of Knowledge in Artificial Intelligence." *Social Studies of Science*, 23(3), 445-477. [Publisher record](#).
- Gruber, T. R. (1993). "A Translation Approach to Portable Ontology Specifications." *Knowledge Acquisition*, 5(2), 199-220. [Publisher record](#).
- Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). "Knowledge Graphs." *ACM Computing Surveys*, 54(4), Article 71. [Open manuscript](#).
- Newell, A. (1982). "The Knowledge Level." *Artificial Intelligence*, 18(1), 87-127. [Publisher record](#)90012-1).
- Nonaka, I. (1994). "A Dynamic Theory of Organizational Knowledge Creation." *Organization Science*, 5(1), 14-37. [Publisher record](#).
- Polanyi, M. (1966). *The Tacit Dimension*. University of Chicago Press.
- Schreiber, G., Akkermans, H., Anjewierden, A., de Hoog, R., Shadbolt, N., Van de Velde, W., and Wielinga, B. (2000). *Knowledge Engineering and Management: The CommonKADS Methodology*. MIT Press. [Publisher record](#).
- Shadbolt, N., and Smart, P. R. (2015). "Knowledge Elicitation: Methods, Tools and Techniques." In J. R. Wilson and S. Sharples, eds., *Evaluation of Human Work*, 4th ed. CRC Press.
- Studer, R., Benjamins, V. R., and Fensel, D. (1998). "Knowledge Engineering: Principles and Methods." *Data & Knowledge Engineering*, 25(1-2), 161-197. [Publisher record](#)00056-6).
- W3C. (2012). *OWL 2 Web Ontology Language Document Overview, Second Edition*. W3C Recommendation. [Specification](#).
- Wielinga, B. J., Schreiber, A. T., and Breuker, J. A. (1992). "KADS: A Modelling Approach to Knowledge Engineering." *Knowledge Acquisition*, 4(1), 5-53. [Publisher record](#)90013-Q).

Document governance note

This paper should evolve through explicit SymK governance. Revisions should distinguish:

- corrections to the history of Knowledge Engineering;
- changes to the conditions interpretation;
- changes to the working definition;
- changes to the epistemic lifecycle;
- changes to candidate foundational commitments;
- changes to development guidance;
- and resolution of open foundational questions.

The lifecycle, object inventory, commitments, and evaluation gate are conceptual proposals. No implementation should silently promote them into the canonical SymK schema. They are constraints and evidence for later vocabulary, ontology, protocol, product, and constitutional work.